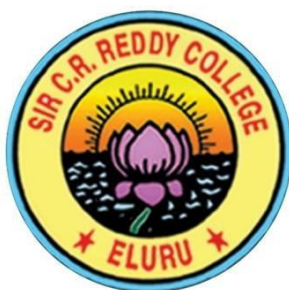


SIR CR REDDY AUTONOMOUS COLLEGE PG COURSES

“SUPERSTORE SALES INSIGHTS ANALYSIS DASHBOARD SYSTEM”

A Project Report Submitted in Partial Fulfillment of the Requirements for the Award of a degree

“Master Of Computer Applications”



Submitted by

MAREEDU KUSUMA SRI

Regd.No. 9240119

Under the esteemed guidance of

Mrs.MEHABOOB KARISHMA

MCA,Mtech(CSE)

Assistant Professor

SIR KATTAMANCHI RAMALINGA REDDY COLLEGE

(AUTONOMOUS)

PG COURSES ELURU

MCA PROGRAMME

Affiliated to AdikaviNannaya University, Rajamahendravaram

Approved by AICTE, Re Accredited at 'A' level by NAAC

2024-2026

DEPARTMENT OF MCA

SIR CR REDDY COLLEGE, ELURU (AUTONOMOUS)



BONAFIDE CERTIFICATE

This is to certify that this Final project Report Entitled is the bonafide work of "SUPERSTORE SALES INSIGHTS ANALYSIS DASHBOARD SYSTEM" submitted by KUSUMA SRI MAREEDU, 9240119 who carried out the work under my guidance and supervision and Submitted in partial fulfillment of the requirements for the award of the MCA during the academic year 2024- 2026.

PROJECT GUIDE

HEAD OF THE DEPARTMENT

Department of MCA

Department of MCA

Submitted for the Project Report Viva-voice held on.....

External Examiner

DECLARATION

I certify that the work contained in this report is original and has been done by me under the guidance of my supervisor. The work has not been submitted to any other Institute for any degree or diploma. I have followed the guidelines provided by the university in preparing the report. I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute. Whenever the used materials(data, theoretical analysis, figures, and text) from other sources, I have given due credit to them by citing them in the text of the report and giving their details in the references. Further, I have taken permission from the copyright owners of the sources, whenever necessary.

ELURU

Date:

MAREEDU KUSUMA SRI

Signature of the student

ACKNOWLEDGEMENTS

*I am highly grateful to Sri **K.SRINIVASRAO**, correspondent, Sir CR Reddy college (A) PG Courses Eluru For inspiring and invaluable encouragement throughout the project*

I profusely thanks **Mr. M RATNAKAR** , Incharge Principal of our college, **SIR C R REDDY COLLEGE, ELURU** for giving permission to carry out this project.

I also thank **Dr. VRS BABU YALAMARTHI**, Director, **SIR C R REDDY COLLEGE, PG COURSES** for making necessary arrangement for doing the project.

I express my sincere thanks to **Mr. M RATNAKAR**, Head of the Department, **SIR CR REDDY COLLEGE, ELURU** for his learned suggestions and encouragement which made this project a successful one.

I am deeply indebted to my project guide **Mrs.MEHABOOB KARISHMA**, Assistant Professor, **SIR C R REDDY COLLEGE, ELURU**, Department of Computer Applications for her valuable guidance, constant encouragement and support throughout the project "SUPERSTORE SALES INSIGHTS ANALYSIS DASHBOARD SYSTEM ".

I would like to thank the review committee members and the entire faculty of Department of Computer Applications for their valuable suggestions and support.

I also express my gratitude to my parents, friends and all those who helped me directly and indirectly in the successful completion of this project.

This project work has been carried out independently by me.

MAREEDU KUSUMA SRI

Regd.No: 9240119

ABSTRACT

Retail organizations generate enormous volumes of transactional data every season, but this data is often scattered across raw CSV exports, spreadsheets, and disconnected reporting tools that make it difficult for stakeholders to extract timely, actionable business insight. This project, “Superstore Sales Analysis insights dashboard system addresses this problem by building a complete, end-to-end analytics pipeline for a fictional retail enterprise — the Superstore — that transforms raw transactional sales data into an interactive, decision-ready Business Intelligence dashboard.

The solution integrates three complementary technologies, each responsible for a distinct stage of the analytics pipeline. Python, using the Pandas and NumPy libraries, is used to acquire and systematically clean the raw dataset — correcting data types, removing duplicate and incomplete records, and validating logical consistency between order and shipping dates. The cleaned dataset is then loaded into a MySQL relational database (`superstore_db`), where fourteen analytical SQL queries — employing aggregation, window functions such as `DENSE_RANK()` and `ROW_NUMBER()`, and Common Table Expressions — are used to answer specific business questions, ranging from identifying top customers by order value to calculating average shipping time and detecting runs of consecutive days of rising sales.

The final stage uses Power BI Desktop to build a three-page interactive report — Sales, Customer, and Product dashboards — each powered by a set of DAX (Data Analysis Expressions) measures computing Key Performance Indicators such as Total Sales, Total Orders, Average Order Value, Total Customers, Customer Lifetime Value, Repeat Customer Rate, and category-wise sales distribution. Interactive slicers for Year, Month, Region, and City allow any stakeholder to explore the data visually and cross-filter every chart on a page without needing to understand the underlying schema or write a single query.

A key strength of this project is its complete, traceable, end-to-end implementation: every insight shown on the final dashboards can be traced back through a specific DAX measure, to a specific cleaned data column, to a specific step of the Python cleaning process — a level of documentation rigour that distinguishes this project from typical single-stage BI demonstrations.

Applied to the Superstore dataset, spanning transactional records from 2015 to 2018, the completed system surfaces meaningful business insight: total sales of approximately \$2 million generated across 4,916 orders at an average order value of \$230; a customer base of 793 individuals with a 99% repeat-purchase rate and a combined customer lifetime value of \$335,000; and a near-even, diversified revenue split across the Technology (36.67%), Furniture (32.11%), and Office Supplies (31.22%) product categories. New York City, Los Angeles, and Seattle emerge as the leading revenue-generating cities, while a small number of

individual products — led by product TEC-CO-10004722 at \$62K — account disproportionately for category-level revenue.

The project demonstrates practical, industry-representative proficiency across the full data analytics lifecycle — data cleaning, relational database design and querying, and interactive Business Intelligence dashboard design — and concludes with a discussion of its limitations and directions for future enhancement, including predictive analytics, automated data refresh, and cloud-based publication of the dashboard via the Power BI Service.

Keywords: Data Analytics, Business Intelligence, Python, Pandas, MySQL, Power BI, DAX, Data Cleaning, Data Visualization, Retail Sales Analysis.

INDEX

(This Table of Contents is an automatically generated field. In Microsoft Word, right-click and choose “Update Field” → “Update entire table” to populate section titles and page numbers.)

CERTIFICATE	Error! Bookmark not defined.
CERTIFICATE ISSUED BY OUTSIDE ORGANIZATION	Error! Bookmark not defined.
ACKNOWLEDGEMENTS.....	Error! Bookmark not defined.
ABSTRACT.....	5
INDEX	7
LIST OF FIGURES.....	10
LIST OF TABLES.....	11
1. INTRODUCTION TO THE PROBLEM	13
1.1 Background	13
1.2 Problem Statement	13
1.3 Existing System	14
1.4 Proposed System	14
1.5 Advantages of the Proposed System.....	15
1.6 Objectives of the Project.....	15
1.7 Scope of the Project	15
1.8 Organization of the Report.....	16
2. STATE OF THE ART / LITERATURE SURVEY	17
2.1 Existing Retail Analytics Approaches	17
2.2 Survey of Data Cleaning and Business Intelligence Tools	17
2.2.1 Python (Pandas and NumPy) for Data Cleaning.....	17
2.2.2 MySQL for Relational Analysis.....	17
2.2.3 Microsoft Power BI	18
2.3 Review of Related Work on Dashboard Design	18
2.4 Comparison of Existing Methods	18
2.5 Gaps Identified in Existing Systems	19
2.6 Summary	19
3. REQUIREMENT ANALYSIS AND DESIGN	20
3.1 Functional Requirements.....	20
3.2 Non-Functional Requirements	20
3.3 Hardware and Software Requirements.....	20
3.4 Feasibility Study.....	21
3.4.1 Technical Feasibility	21

3.4.2 Economic Feasibility	21
3.4.3 Operational Feasibility	21
3.5 Data Sources and Dataset Description	21
3.6 System Architecture.....	22
3.7 Workflow Diagram	22
3.8 Data Flow Diagram (Level 0 and Level 1).....	23
3.8.1 Level 0 DFD (Context Diagram)	23
3.8.2 Level 1 DFD.....	24
3.8.3 External Entities, Processes, and Data Stores	25
3.9 Use Case Diagram	25
3.10 ER Diagram / Data Model	26
3.11 Database Design	26
3.12 Data Dictionary	26
3.13 Dashboard Page Design	27
3.13.1 Sales Dashboard	27
3.13.2 Customer Dashboard	27
3.13.3 Product Dashboard	27
3.14 Navigation and UI/UX Considerations	27
4. IMPLEMENTATION	29
4.1 Tools and Technologies Used	29
4.2 Module 1: Data Collection	29
4.3 Module 2: Data Cleaning using Python.....	30
4.4 Module 3: SQL Database Creation	34
4.5 Module 4: SQL Queries and Business Insights	35
4.5.1 Q1 – Same-Day Shipping Percentage	36
4.5.2 Q2 – Top 3 Customers by Total Order Value	36
4.5.3 Q3 – Top 5 Items by Average Daily Sales.....	37
4.5.4 Q4 – Customer Ranking by Average Order Value	37
4.5.5 Q5 – Highest and Lowest Orders per City	37
4.5.6 Q6 – Most Demanded Sub-Category in the West Region	38
4.5.7 Q7 – Order With the Highest Number of Items	38
4.5.8 Q8 – Order With the Highest Cumulative Value	38
4.5.9 Q9 – Segment Most Likely to Use First Class Shipping	39
4.5.10 Q10 – City Contributing Least to Total Revenue.....	39
4.5.11 Q11 – Average Order-to-Shipment Time	39
4.5.12 Q12 – Dominant Segment per State.....	39

4.5.13 Q13 – Customers Ordering on 3 Consecutive Days (> \$50)	40
4.5.14 Q14 – Maximum Streak of Rising Daily Sales	40
4.6 Module 5: Power BI Dashboard Development	41
4.6.2 Sales Dashboard	43
4.6.3 Customer Dashboard	43
4.6.4 Product Dashboard	43
4.7 Module 6: Visualization and KPI Analysis	43
4.8 Navigation and Interactivity	44
5. RESULTS ALONG WITH TEST CASES	45
5.1 Dashboard Output	45
5.1.1 Sales Dashboard Page	45
5.1.2 Customer Dashboard Page	46
5.1.3 Product Dashboard Page	47
5.2 Region Analysis	48
5.3 Summary of Sample KPI Outputs	48
5.4 Test Cases and Results	48
5.5 Discussion of Results	49
6. CONCLUSIONS AND FUTURE WORK	50
6.1 Conclusion	50
6.2 Limitations	50
6.3 Future Enhancements	51
7. BIBLIOGRAPHY	52

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 1	System Architecture of the Superstore Sales Analysis System	23
Figure 2	Project Workflow Diagram	24
Figure 3	Data Flow Diagram – Level 0 (Context Diagram)	25
Figure 4	Data Flow Diagram – Level 1	26
Figure 5	Use Case Diagram	26
Figure 6	Entity-Relationship Diagram / Logical Data Model	27
Figure 7	Algorithm Flowchart – Data Cleaning Process	32
Figure 8	Raw Dataset – Sample View (sales_df.head() output)	31
Figure 9	Python Data Cleaning – Null Value Check Output	33
Figure 10	Python Data Cleaning – Duplicate Record Removal Output	34
Figure 11	MySQL Table Structure – superstore (DESCRIBE output)	35
Figure 12	Algorithm Flowchart – SQL Query Processing	36
Figure 13	SQL Query Output – Top 3 Customers by Order Value	38
Figure 14	Algorithm Flowchart – Power BI Dashboard Development	42
Figure 15	Power BI Data Model View	43
Figure 16	Home Page – Power BI Dashboard (Navigation)	44
Figure 17	Sales Dashboard – Final Output	46
Figure 18	Customer Dashboard – Final Output	47
Figure 19	Product Dashboard – Final Output	48

LIST OF TABLES

Table No.	Title	Page No.
Table 1	Comparison of Existing Retail Analysis Methods	19
Table 2	Hardware and Software Requirements	21
Table 3	Dataset Description and Source Details	22
Table 4	Data Dictionary – superstore Table	27
Table 5	Summary of SQL Business Insight Queries	36
Table 6	Representative DAX Measures used in the Dashboard	43

LIST OF OUTPUT TABLES

Table No.	Title	Page No.
Output Table 1	Key Business Insights – Sales Dashboard	46
Output Table 2	Key Business Insights – Customer Dashboard	47
Output Table 3	Key Business Insights – Product Dashboard	49
Output Table 4	Test Cases and Results	49

1. INTRODUCTION TO THE PROBLEM

1.1 Background

In the contemporary retail industry, data has become one of the most valuable assets an organization possesses. Every transaction — a customer placing an order, a product being shipped, a discount being applied — generates a data point that, when aggregated with millions of others, reveals patterns about customer behaviour, product performance, and regional demand. Retail enterprises that can effectively capture, process, and interpret this data hold a significant competitive advantage over those that rely purely on intuition or delayed, static reporting.

This project focuses on the design and implementation of a complete sales analytics solution for a fictional retail enterprise referred to as the “Superstore”. The Superstore dataset is a widely used, publicly available dataset in the data analytics community, containing several thousand order-level transactional records spanning multiple years (2015–2018), product categories, customer segments, and geographic regions across the United States. Although fictional, the dataset closely mirrors the structure and characteristics of real-world retail transaction data, making it an appropriate and realistic basis for demonstrating a complete analytics workflow.

Business Intelligence (BI) platforms such as Microsoft Power BI make it possible to transform such raw tabular data into interactive, visual, and explorable dashboards without requiring every stakeholder to possess programming or query-writing skills. Combined with Python for data preparation and SQL for structured querying, a small but well-designed technology stack can support the entire journey from raw CSV file to executive-ready dashboard.

1.2 Problem Statement

Raw retail transaction data, in its native form, is rarely suitable for direct business use. It is often voluminous, may contain missing or inconsistent values, duplicate entries, and incorrectly typed fields (for example, dates stored as text). Furthermore, even a perfectly clean dataset, if left in flat tabular form, does not by itself communicate business insight — stakeholders cannot easily determine which regions are underperforming, which products are most profitable, or how customer behaviour is trending over time simply by viewing rows of transactional data.

The problem this project addresses can therefore be stated as follows: given a raw, uncleaned retail sales dataset, how can that data be systematically transformed — through cleaning, structured querying, and visualization — into an interactive decision-support system that allows business stakeholders to explore

sales performance, customer behaviour, and product trends without requiring technical knowledge of the underlying data or query languages?

1.3 Existing System

In many small and medium-sized retail organizations, sales analysis continues to be performed using manual, spreadsheet-based methods. Sales data is typically exported from a point-of-sale or order-management system into Microsoft Excel, where analysts manually construct pivot tables and static charts to summarise performance. While functional for small datasets, this approach suffers from several well-documented limitations:

- Static reports quickly become outdated and must be manually regenerated whenever new data becomes available.
- Spreadsheet-based analysis does not scale well beyond a few thousand rows and becomes slow and error-prone as data volume grows.
- Formulas and pivot table configurations are difficult to audit, version, and reuse, increasing the risk of undetected calculation errors.
- Cross-filtering between multiple dimensions (for example, viewing sales for a specific region, category, and time period simultaneously) is cumbersome and often requires separate, disconnected pivot tables.
- There is no single interactive interface through which non-technical stakeholders can explore the data themselves; they must instead rely on analysts to generate new reports on request.

1.4 Proposed System

The proposed system replaces the manual, spreadsheet-based approach with an automated, three-tier analytics pipeline. Data cleaning is performed programmatically using Python scripts that can be re-run consistently on new data extracts, eliminating manual, error-prone spreadsheet manipulation. The cleaned data is stored in a structured MySQL database, enabling powerful, reusable SQL queries that can be executed instantly rather than reconstructed from scratch. Finally, Power BI is used to build a live, interactive dashboard that allows any stakeholder to filter, drill down, and explore the data visually, without needing to write queries or understand the underlying schema.

This proposed architecture separates the concerns of data preparation, data storage and querying, and data presentation into distinct, well-defined stages, which is consistent with standard practice in modern

Business Intelligence system design, and is described in full detail in Chapter 3 (Requirement Analysis and Design) and Chapter 4 (Implementation).

1.5 Advantages of the Proposed System

- Automated, repeatable data cleaning reduces manual effort and the risk of human error.
- Structured storage in MySQL supports fast, reliable, and reusable querying compared to spreadsheet formulas.
- Interactive Power BI dashboards allow non-technical users to explore data through visual filters and slicers.
- Clear separation between data preparation, storage, and visualization layers improves maintainability.
- KPI-driven dashboard design ensures stakeholders can immediately grasp overall business performance at a glance.
- The modular pipeline can be extended in the future to support new data sources, additional KPIs, or predictive analytics.

1.6 Objectives of the Project

- To acquire and thoroughly clean the Superstore sales dataset using Python, ensuring correct data types, no missing critical values, and no duplicate records.
- To design and implement a relational database schema in MySQL for storing the cleaned transactional data.
- To write and execute SQL queries that extract specific, decision-relevant business insights from the dataset.
- To design an interactive, multi-page Power BI dashboard covering Sales, Customer, and Product analysis perspectives.
- To compute and present key business KPIs, including Total Sales, Total Orders, Average Order Value, Total Customers, Customer Lifetime Value, and Sales Trends, using DAX measures.
- To derive and document meaningful business insights from the completed analysis.
- To demonstrate an end-to-end, industry-representative data analytics workflow suitable for academic evaluation.

1.7 Scope of the Project

The scope of this project encompasses the complete analytics lifecycle applied to a static, historical retail dataset: data acquisition, cleaning and preprocessing, relational database design, SQL-based analysis, and dashboard-based visualization. The project covers three analytical dimensions — sales performance, customer behaviour, and product performance — each represented by a dedicated dashboard page, filterable by year, month, region, and city.

The scope of this project does not extend to real-time or streaming data ingestion, predictive modelling or machine-learning-based forecasting, multi-user authentication or role-based access control within the dashboard, or deployment on cloud-hosted BI services such as the Power BI Service. These aspects are identified as potential directions for future enhancement in Chapter 6.

1.8 Organization of the Report

The remainder of this report is organized as follows. Chapter 2 presents a literature survey of existing approaches to retail sales analytics and Business Intelligence tooling, identifying the research gap this project addresses. Chapter 3 details the requirement analysis and system design, including functional and non-functional requirements, feasibility study, system architecture, workflow, data flow diagrams, use case diagram, entity-relationship diagram, database design, and data dictionary. Chapter 4 describes the implementation of each module of the system, including data cleaning, database creation, SQL analysis, and Power BI dashboard development, along with relevant code, algorithms, and flowcharts. Chapter 5 presents the results obtained from the completed system, along with test cases and their outcomes. Chapter 6 concludes the report with a summary of achievements, a discussion of limitations, and recommendations for future work. The report closes with a bibliography listing all references consulted during the course of this project.

2. STATE OF THE ART / LITERATURE SURVEY

2.1 Existing Retail Analytics Approaches

Several commercial and open-source tools currently provide retail and e-commerce analytics capabilities. Enterprise Resource Planning (ERP) systems such as SAP and Oracle NetSuite include built-in sales reporting modules, but these are typically expensive, complex to configure, and oriented toward transactional record-keeping rather than exploratory, interactive analysis. At the other end of the spectrum, spreadsheet-based analysis using Microsoft Excel remains the most common approach in small and medium enterprises, owing to its low cost and familiarity, despite the scalability and maintainability limitations discussed in Section 1.3.

In academic and hobbyist circles, numerous tutorials and open-source projects use the Superstore dataset specifically — owing to its public availability, realistic structure, and moderate size — as a benchmark example for demonstrating individual data analytics skills. However, these typically demonstrate isolated stages of a pipeline (for example, only a Power BI dashboard built directly from the CSV file, or only a set of SQL queries) rather than a fully integrated, end-to-end workflow that carries the data through cleaning, structured storage, querying, and visualization with full traceability.

2.2 Survey of Data Cleaning and Business Intelligence Tools

Three categories of tools were surveyed and evaluated for suitability in this project: Python-based data cleaning libraries, relational database systems, and self-service Business Intelligence platforms.

2.2.1 Python (Pandas and NumPy) for Data Cleaning

The Python data science ecosystem, particularly the Pandas and NumPy libraries, has become a widely adopted standard for data cleaning, transformation, and exploratory analysis. Pandas provides a DataFrame abstraction that closely mirrors a relational table, along with vectorised operations for handling missing values (`isnull()`, `dropna()`), duplicate detection (`duplicated()`, `drop_duplicates()`), and type conversion (`astype()`, `to_datetime()`), all of which were used extensively in this project's data cleaning module. Its popularity, extensive documentation, and tight integration with Jupyter Notebook made it the natural choice for this project's cleaning stage.

2.2.2 MySQL for Relational Analysis

Structured Query Language (SQL) remains the standard mechanism for extracting aggregated, filtered, and ranked information from relational datasets. MySQL, an open-source relational database management system, was selected for this project due to its free licensing, wide industry adoption, strong support for

modern SQL features including window functions (RANK(), DENSE_RANK(), ROW_NUMBER()) and Common Table Expressions (CTEs), and native connectivity with Power BI.

2.2.3 Microsoft Power BI

Among self-service Business Intelligence platforms — which also include Tableau and Qlik Sense — Power BI Desktop was selected for this project because it is free for individual and academic use, offers a low barrier to entry through its drag-and-drop visual design surface, and provides the DAX (Data Analysis Expressions) formula language for authoring calculated KPI measures. Its tight integration with both CSV/Excel files and MySQL databases (via native connectors) made it well suited to consuming the output of the earlier Python and SQL stages of this project's pipeline.

2.3 Review of Related Work on Dashboard Design

The design of the three dashboard pages in this project draws on established dashboard design literature, which consistently emphasises that effective BI dashboards should minimise cognitive load by presenting a small number of clearly labelled Key Performance Indicator (KPI) cards alongside a limited set of well-chosen visualizations, rather than overwhelming the user with excessive detail. This principle directly informed the layout of the Sales, Customer, and Product dashboard pages, each of which places two to three headline KPI cards prominently at the top of the page, followed by supporting bar, donut, and trend charts.

Literature on relational data modelling for analytics — particularly the concept of dimensional modelling and the star schema, in which a central fact table (transactional records) is linked to supporting dimension tables (customer, product, location, date) — also informed the logical entity-relationship model presented in Chapter 3, even though the physical database implementation in this project uses a single denormalised table optimised for direct Power BI import.

2.4 Comparison of Existing Methods

Table 1 compares the manual spreadsheet-based approach, a SQL-only reporting approach, and the integrated Python–SQL–Power BI approach adopted in this project, across several relevant evaluation criteria.

Table 1: Comparison of Existing Retail Analysis Methods

Criteria	Spreadsheet-Based Analysis	SQL-Only Reporting	Proposed Python + SQL + Power BI
Scalability	Poor beyond a few thousand rows	Good	Good

Criteria	Spreadsheet-Based Analysis	SQL-Only Reporting	Proposed Python + SQL + Power BI
Data Cleaning Automation	Manual, error-prone	Limited	Automated via Python scripts
Reusability of Queries	Low	High	High
Interactivity for End Users	Very Limited	None (requires query knowledge)	High (slicers, drill-down)
Visualization Quality	Basic charts	None (raw output only)	Rich, interactive visuals
Ease of Use for Non-Technical Users	Moderate	Low	High

As shown in Table 1, the proposed integrated approach combines the scalability and reusability benefits of SQL with the accessibility and interactivity of a modern BI tool, while automating the otherwise manual and error-prone data cleaning stage — addressing the principal weaknesses of both alternative approaches.

2.5 Gaps Identified in Existing Systems

- Existing examples using the Superstore dataset rarely carry the data through all three stages (Python cleaning, MySQL storage and querying, and Power BI visualization) with full documentation and traceability from raw data to final insight.
- Existing dashboards commonly focus only on Sales-level metrics and omit a dedicated Customer Dashboard, despite metrics such as Customer Lifetime Value and Repeat Customer Rate being of significant business importance.
- Data cleaning steps are frequently under-documented or skipped entirely in published examples, leaving the reliability of the resulting KPIs unverifiable.
- Few examples make explicit use of SQL window functions and Common Table Expressions, relying instead on simple aggregation queries that cannot answer more advanced business questions such as customer ranking or identifying runs of rising sales.

2.6 Summary

The literature and platform survey confirms that while individual pieces of the problem — Python-based data cleaning, SQL-based querying, and Power BI-based visualization — have each been solved elsewhere in isolation, few existing examples combine all three into a single, fully documented, end-to-end pipeline with a dedicated, drill-down-capable Customer Dashboard alongside Sales and Product dashboards. This project addresses that gap by implementing and documenting a complete, three-stage pipeline with three purpose-built dashboard pages, each targeting a distinct business perspective, as detailed in Chapters 3 and 4.

3. REQUIREMENT ANALYSIS AND DESIGN

3.1 Functional Requirements

- FR1: The system shall import raw sales data from a CSV source file.
- FR2: The system shall clean the raw data by correcting data types, removing duplicate records, and handling missing values.
- FR3: The system shall export the cleaned dataset and load it into a MySQL relational database.
- FR4: The system shall support execution of SQL queries to compute business insights such as top customers, top products, and regional performance.
- FR5: The system shall present a Sales Dashboard displaying total sales, total orders, average order value, top cities, product category distribution, and sales trend over time.
- FR6: The system shall present a Customer Dashboard displaying total customers, customer lifetime value, repeat customer rate, and segment-wise distribution.
- FR7: The system shall present a Product Dashboard displaying total products, category-wise sales, and top-selling products.
- FR8: The system shall allow users to filter dashboard visuals by year, month, region, and city using interactive slicers.

3.2 Non-Functional Requirements

- Usability: The dashboard interface must be intuitive enough for a non-technical business user to operate without training.
- Performance: Dashboard visuals must refresh (cross-filter) within a few seconds of a filter or slicer being applied.
- Maintainability: The data cleaning scripts and SQL queries must be modular and well-commented to support future modification.
- Portability: The Power BI report file (.pbix) must be shareable and viewable on any machine with Power BI Desktop installed.
- Reliability: Data cleaning logic must produce consistent, reproducible results when re-run on the same input data.

3.3 Hardware and Software Requirements

Table 2: Hardware and Software Requirements

Requirement	Specification
Processor	Intel Core i3 (or equivalent) or higher
RAM	8 GB or higher (recommended for Power BI Desktop)
Hard Disk	10 GB free disk space
Display	1366 x 768 resolution or higher
Operating System	Windows 10 / 11 (64-bit)
Programming Language	Python 3.x
Python Libraries	Pandas, NumPy, Matplotlib, Seaborn
Database Server	MySQL 8.0
Visualization Tool	Power BI Desktop
Development Environment	Jupyter Notebook

3.4 Feasibility Study

3.4.1 Technical Feasibility

All tools used in this project — Python, MySQL, and Power BI Desktop — are freely available (Power BI Desktop is free for individual use) and widely documented, with the developer having prior working knowledge of each. The Superstore dataset is small enough (a few thousand rows) to be processed comfortably on standard consumer hardware, making the project technically feasible within the available time and resource constraints.

3.4.2 Economic Feasibility

The project incurs no licensing cost, as MySQL Community Edition and Power BI Desktop are both free to use for academic and individual purposes. The only resource investment required is developer time, making the project highly economical to execute.

3.4.3 Operational Feasibility

The resulting dashboard is designed to be self-explanatory and interactive, requiring no specialised training for end users to interpret KPIs or apply filters. This ensures strong operational feasibility, as the system can realistically be adopted by non-technical business stakeholders.

3.5 Data Sources and Dataset Description

The dataset used in this project is the Superstore Sales Dataset, a widely used, publicly available retail transaction dataset covering the years 2015–2018. It was obtained as a single CSV file and cleaned prior to being loaded into MySQL, as described fully in Chapter 4.

Table 3: Dataset Description and Source Details

Dataset	Key Columns	Source	Role
Superstore Raw Data	Order_ID, Order_Date, Ship_Date, Customer_ID, Product_ID, Sales, ...	Public CSV export (superstore_final_dataset.csv)	Raw input to Python cleaning stage
Superstore Cleaned Data	Same schema, cleaned and type-corrected	Output of Python cleaning notebook	Source for MySQL import
superstore (MySQL table)	18 typed columns (see Data Dictionary, Table 3)	Loaded from cleaned CSV	Source for SQL analysis and Power BI

3.6 System Architecture

The system follows a linear, three-tier pipeline architecture consisting of a data preparation tier (Python), a data storage and querying tier (MySQL), and a visualization tier (Power BI). Figure 1 illustrates how raw data flows through each tier to produce the three final dashboard pages.

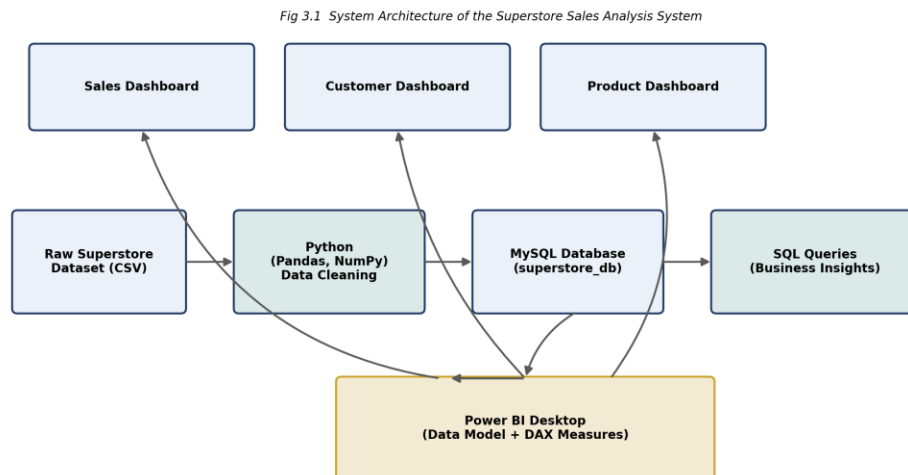


Figure 1: System Architecture of the Superstore Sales Analysis System

3.7 Workflow Diagram

Figure 2 presents the step-by-step workflow followed during the execution of this project, from initial data collection through to final dashboard publication and analysis.

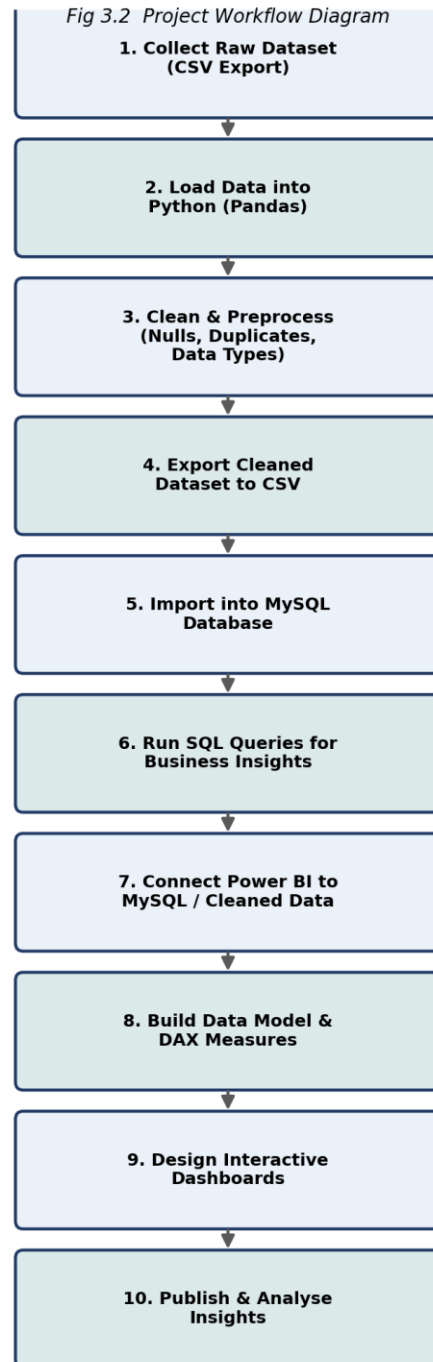


Figure 2: Project Workflow Diagram

3.8 Data Flow Diagram (Level 0 and Level 1)

3.8.1 Level 0 DFD (Context Diagram)

The Level 0 Data Flow Diagram, shown in Figure 3, presents the system as a single process, showing the flow of data between the Business Analyst / User, the Superstore Sales Analysis System, and the MySQL database.

Fig 3.3 Data Flow Diagram – Level 0 (Context Diagram)



Figure 3: Data Flow Diagram – Level 0 (Context Diagram)

3.8.2 Level 1 DFD

The Level 1 Data Flow Diagram, shown in Figure 4, decomposes the system into its four principal processes — Data Cleaning, Data Loading, Query Processing, and Visualization — along with the associated data stores for the cleaned dataset and the MySQL database table.

Fig 3.4 Data Flow Diagram – Level 1

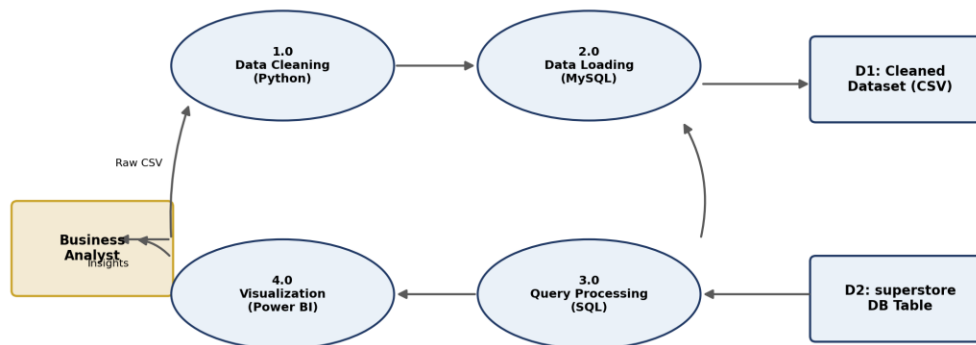


Figure 4: Data Flow Diagram – Level 1

3.8.3 External Entities, Processes, and Data Stores

External Entities: Business Analyst / User — supplies filters and slicer selections and receives the finished dashboard; supplies no raw data directly to the pipeline.

Processes: 1.0 Data Cleaning (Python) reads the raw CSV and writes the cleaned dataset; 2.0 Data Loading (MySQL) reads the cleaned dataset and writes to the database table; 3.0 Query Processing (SQL) reads from the database table and produces query results; 4.0 Visualization (Power BI) reads query results / cleaned data and renders the interactive dashboard.

Data Stores: D1 Cleaned Dataset (CSV) — written by Process 1.0, read by Process 2.0; D2 superstore DB Table — written by Process 2.0, read by Process 3.0.

3.9 Use Case Diagram

Figure 5 identifies the two principal actors of the system — the Data Analyst / Developer, responsible for data preparation and dashboard construction, and the Business User, who consumes the finished dashboards — along with their respective use cases.

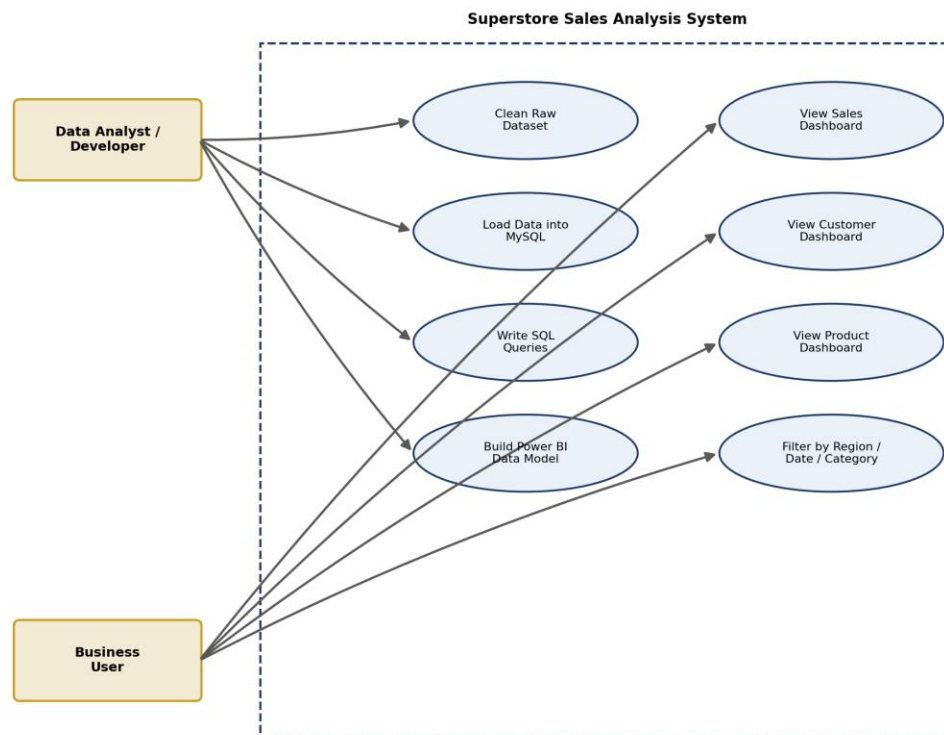


Fig 3.5 Use Case Diagram

Figure 5: Use Case Diagram

3.10 ER Diagram / Data Model

Although the physical implementation stores data in a single denormalised table (optimised for analytical querying and Power BI import), Figure 6 presents the logical entity-relationship model underlying the dataset, decomposed into Customer, Order, Product, Location, and Sales_Line entities for conceptual clarity.

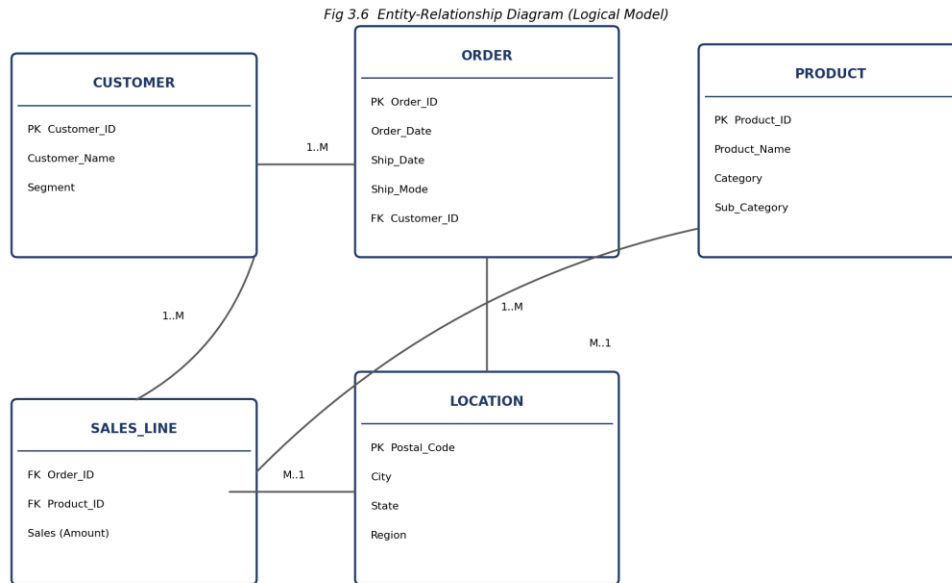


Figure 6: Entity-Relationship Diagram / Logical Data Model

3.11 Database Design

The physical database implementation uses a single table, superstore, within the superstore_db MySQL database. This denormalised, “flat” table design is a deliberate choice appropriate for analytical (OLAP-style) workloads, where join operations across multiple normalized tables would add unnecessary query complexity and reduce performance for read-heavy, aggregation-focused queries — a pattern consistent with typical BI staging tables that feed directly into Power BI's own internal data model.

3.12 Data Dictionary

Table 4 documents each column of the superstore table, including its data type and business meaning, as derived directly from the project's schema definition.

Table 4: Data Dictionary – superstore Table

Field Name	Data Type	Description
Order_ID	varchar(14)	Unique identifier for a customer order

Field Name	Data Type	Description
Order_Date	varchar(10)	Date on which the order was placed
Ship_Date	varchar(10)	Date on which the order was shipped
Ship_Mode	varchar(14)	Shipping method (e.g., Standard Class, First Class)
Customer_ID	varchar(8)	Unique identifier for the customer
Customer_Name	varchar(22)	Full name of the customer
Segment	varchar(11)	Customer segment (Consumer, Corporate, Home Office)
Country	varchar(13)	Country of the order (United States)
City	varchar(17)	City associated with the order
State	varchar(20)	State associated with the order
Postal_Code	int	Postal code of the delivery location
Region	varchar(7)	Geographic region (West, East, Central, South)
Product_ID	varchar(15)	Unique identifier for the product
Category	varchar(15)	Product category (Furniture, Office Supplies, Technology)
Sub_Category	varchar(11)	Product sub-category (e.g., Chairs, Binders, Phones)
Product_Name	varchar(127)	Full descriptive name of the product
Sales	decimal(20,10)	Sales amount (in USD) for the order line

3.13 Dashboard Page Design

3.13.1 Sales Dashboard

Designed as a top-level command centre: Total Sales, Total Orders, and Average Order Value KPI cards at the top; a Top 10 City by Sales bar chart and Sales Distribution by Product Category donut chart in the middle; and a Sales Trend line chart across the bottom, filterable by Year and Month slicers.

3.13.2 Customer Dashboard

Designed around customer-level metrics: Total Customers, Customer Lifetime Value, and Repeat Customer Rate KPI cards at the top; a Top 10 City Segment Distribution by Sales stacked bar chart and Customer Distribution by Segment donut chart; and a Customer LTV by Region bar chart.

3.13.3 Product Dashboard

Designed around product-level metrics: Total Sales and Total Number of Products KPI cards; a Sales by Product Category donut chart and Sales Trend of Product Category Over Time stacked column chart; and a Top 10 Selling Products by Sales horizontal bar chart.

3.14 Navigation and UI/UX Considerations

All three dashboard pages share a consistent visual language — the same colour palette, font, and KPI card styling — so that a user moving between the Sales, Customer, and Product pages experiences a coherent, unified report rather than three disconnected views. Page tabs at the top of the Power BI report allow direct navigation between pages, and slicers for Year, Month, Region, and City are positioned consistently in the top-right corner of each page so that a filter selection is easy to locate and modify regardless of which page the user is currently viewing.

4. IMPLEMENTATION

This chapter describes the implementation of the Superstore Sales Analysis system in detail, organised into six modules corresponding to the successive stages of the data pipeline: Data Collection, Data Cleaning using Python, SQL Database Creation, SQL Queries and Business Insights, Power BI Dashboard Development, and Visualization and KPI Analysis.

4.1 Tools and Technologies Used

- Python 3.x — primary language for data acquisition and cleaning.
- Pandas and NumPy — libraries used for DataFrame manipulation, type conversion, and numerical operations.
- Jupyter Notebook — interactive development environment used for the data cleaning workflow.
- MySQL 8.0 — relational database server used to store the cleaned dataset and execute analytical SQL queries.
- Microsoft Power BI Desktop — primary tool used for data modelling, DAX measure authoring, and dashboard visualization.
- DAX (Data Analysis Expressions) — formula language used to author all KPI measures in Power BI.

4.2 Module 1: Data Collection

The dataset used in this project is the Superstore Sales Dataset, obtained as a raw CSV file (superstore_final_dataset.csv) containing 18 columns, including order identifiers, order and shipping dates, shipping mode, customer details, geographic details, product details, and the sales amount for each order line.

The dataset was loaded into a Python environment (Jupyter Notebook) using the Pandas library for subsequent inspection and cleaning:

```
# Importing necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime

import warnings
warnings.filterwarnings('ignore')
```

```
# Reading data from CSV file
sales_df = pd.read_csv('superstore_final_dataset.csv')
```

Code 4.1: Importing Libraries and Loading the Raw Dataset

Initial inspection of the dataset was performed using `sales_df.head()`, `sales_df.shape`, and `sales_df.info()` to understand the structure, size, and data types of the raw data before proceeding to the cleaning stage.

	A	B	C	D	E	F	G	H	I
1	Order ID	Order Date	Ship Mode	Customer Name	Segment	Product	Sales	Quantity	Profit
2	CA-2019-1001	01/06/2019	Second Class	Paul Allen	Consumer	Canon Copier	\$1,408.00	1	#N/A
3	US-2020-1588			#N/A	Corporate	Staples Paper C	\$15.04	3	
4		02/14/2020		#N/A				3	
5		03-02-21*	First Class	Steve Hill	#N/A	HP Lizerzjet	\$335.04		
6	143675	??/18/2020	Standard Class	#N/A	Consumer	Ballpoint Pen	#VALUE!	2	+VALUE!
7				#N/A				2	
8		05/22/2018	SameDay		Corporate		9995.00	1	NA
9	143456	#N/A	Standard Class	Monika Patel	Consumer	??Binder		4	#DIV/0!
10				#N/A				4	
11	Ch-12072	07/29/2021			Home Office	iPad Air	\$678.00	1	???

Figure 8: Raw Dataset – Sample View (`sales_df.head()` output)

4.3 Module 2: Data Cleaning using Python

Data cleaning is a critical stage of any analytics pipeline, as downstream SQL analysis and dashboard visuals are only as reliable as the underlying data. The cleaning process for this project, implemented in the accompanying Jupyter Notebook, followed a systematic, verifiable sequence of algorithmic steps, illustrated as a flowchart below.

Fig 4.10 Algorithm Flowchart – Data Cleaning Process

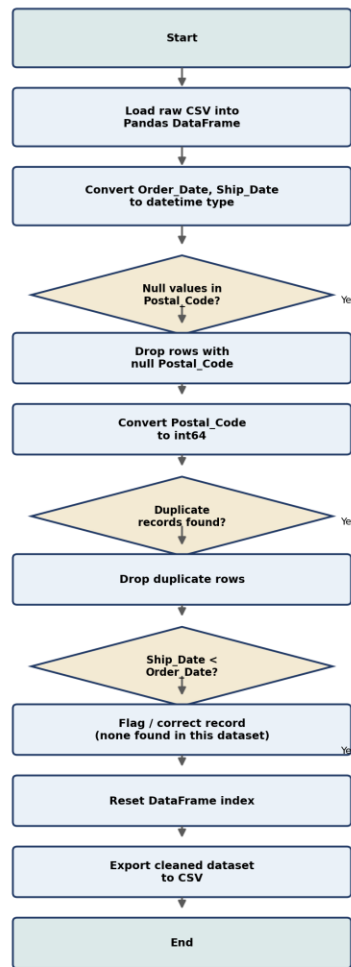


Figure 7: Algorithm Flowchart – Data Cleaning Process

The date columns were converted from text to proper datetime objects to enable correct chronological sorting and duration calculations:

```
# Changing the data type of 'Order_Date' and 'Ship_Date' to datetime type
sales_df['Order_Date'] = pd.to_datetime(sales_df['Order_Date'], format="%d/%m/%Y")
sales_df['Ship_Date'] = pd.to_datetime(sales_df['Ship_Date'], format="%d/%m/%Y")
```

Code 4.2: Converting Date Columns to Datetime Type

A null-value check revealed that only the Postal_Code column contained missing values, and that these missing records represented a very small fraction of the overall dataset. Given their limited number, the affected rows were removed rather than imputed, to avoid introducing artificial postal code values that could distort city/state-level aggregation:

```
# Checking the null values
sales_df.isnull().sum()

# Dropping rows with null values in Postal_Code
sales_df = sales_df[~(sales_df['Postal_Code'].isnull())]

# Converting the dtype of Postal_Code from float64 to int64
sales_df['Postal_Code'] = sales_df['Postal_Code'].astype('int64')
```

Code 4.3: Handling Missing Postal_Code Values

Order_ID	Order_Date	Ship_Date	Ship_Mode	Customer	Customer_Segment	Country	City	State	Postal_Code	Region	Product_ID	Category	Sub_Category	Product_Name	Sales
CA-2017-1	2017-01-01	2017-01-01	Standard	Claire Gutierrez	Consumer	United States	Henderson	Kentucky	42420	South	FUR-BO-1F	Furniture	Bookcases	Bush Somerset	261.96
CA-2017-1	2017-01-01	2017-01-01	Standard	Claire Gutierrez	Consumer	United States	Henderson	Kentucky	42420	South	FUR-CH-1F	Furniture	Chairs	Honore Delux	731.94
CA-2017-1	2017-01-01	2017-01-01	Standard	Darrin Van Cate	Corporate	United States	Los Angeles	California	90036	West	OFF-LA-1O	Office Supplies	Labels	Self-Adhesive	14.62
US-2016-1	2016-01-01	2016-01-01	Standard	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	FUR-TA-1C	Furniture	Tables	Bretford Classic	957.5775
US-2016-1	2016-01-01	2016-01-01	Standard	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	OFF-ST-1O	Office Supplies	Storage	Eldon Fold	22.368
CA-2015-1	2015-01-01	2015-01-01	Standard	Brosina H. Consumer	Consumer	United States	Los Angeles	California	90032	West	FUR-FU-1C	Furniture	Furnishings	Eldon Express	48.86
CA-2015-1	2015-01-01	2015-01-01	Standard	Brosina H. Consumer	Consumer	United States	Los Angeles	California	90032	West	OFF-AR-1C	Office Supplies	Art	Newell 32	7.28
CA-2015-1	2015-01-01	2015-01-01	Standard	Brosina H. Consumer	Consumer	United States	Los Angeles	California	90032	West	TEC-PH-1C	Technology	Phones	Mitel 5320	907.152
CA-2015-1	2015-01-01	2015-01-01	Standard	Brosina H. Consumer	Consumer	United States	Los Angeles	California	90032	West	OFF-BI-1O	Office Supplies	Binders	DXL Angle	18.504
CA-2015-1	2015-01-01	2015-01-01	Standard	Brosina H. Consumer	Consumer	United States	Los Angeles	California	90032	West	OFF-AP-1O	Office Supplies	Appliances	Belkin F5C	114.9
CA-2015-1	2015-01-01	2015-01-01	Standard	Brosina H. Consumer	Consumer	United States	Los Angeles	California	90032	West	FUR-TA-1C	Furniture	Tables	Chromcraft	1706.184
CA-2015-1	2015-01-01	2015-01-01	Standard	Brosina H. Consumer	Consumer	United States	Los Angeles	California	90032	West	TEC-PH-1C	Technology	Phones	Konitel 25	911.424
CA-2018-1	2018-01-01	2018-01-01	Standard	Andrew All	Consumer	United States	Concord	North Carolina	28027	South	OFF-PA-1O	Office Supplies	Paper	Xerox 196	15.552
CA-2017-1	2017-01-01	2017-01-01	Standard	Irene Mad	Consumer	United States	Seattle	Washington	98103	West	OFF-BI-1O	Office Supplies	Binders	Fellowes P	407.976
US-2016-1	2016-01-01	2016-01-01	Standard	Harold Pax	Home Office	United States	Fort Worth	Texas	76106	Central	OFF-AP-1O	Office Supplies	Appliances	Holmes Re	68.81
US-2016-1	2016-01-01	2016-01-01	Standard	Harold Pax	Home Office	United States	Fort Worth	Texas	76106	Central	OFF-BI-1O	Office Supplies	Binders	Stores Dur	2.544
CA-2015-1	2015-01-01	2015-01-01	Standard	Pete Kriz	Consumer	United States	Madison	Wisconsin	53711	Central	OFF-ST-1O	Office Supplies	Storage	Stur-D-Sto	665.88
CA-2015-1	2015-01-01	2015-01-01	Standard	Alejandro C	Consumer	United States	West Jordan	Utah	84064	West	OFF-ST-1O	Office Supplies	Storage	Fellowes S	55.5
CA-2015-1	2015-01-01	2015-01-01	Standard	Zuschuss C	Consumer	United States	San Francisco	California	94109	West	OFF-AR-1C	Office Supplies	Art	Newell 34	8.56
CA-2015-1	2015-01-01	2015-01-01	Standard	Zuschuss C	Consumer	United States	San Francisco	California	94109	West	TEC-PH-1C	Technology	Phones	Cisco SPA	213.48
CA-2015-1	2015-01-01	2015-01-01	Standard	Zuschuss C	Consumer	United States	San Francisco	California	94109	West	OFF-BI-1O	Office Supplies	Binders	Wilson Jor	22.72
CA-2017-1	2017-01-01	2017-01-01	Standard	Ken Black	Corporate	United States	Fremont	Nebraska	68025	Central	OFF-AR-1C	Office Supplies	Art	Newell 31	19.46
CA-2017-1	2017-01-01	2017-01-01	Standard	Ken Black	Corporate	United States	Fremont	Nebraska	68025	Central	OFF-AP-1O	Office Supplies	Appliances	Acco Six-G	60.34
US-2018-1	2018-01-01	2018-01-01	Standard	Sandra Fla	Consumer	United States	Philadelphia	Pennsylvania	19140	East	FUR-CH-1F	Furniture	Chairs	Global Del	71.372
CA-2016-1	2016-01-01	2016-01-01	Standard	Emily Burn	Consumer	United States	Orem	Utah	84057	West	FUR-TA-1C	Furniture	Tables	Bretford C	1044.63
CA-2017-1	2017-01-01	2017-01-01	Standard	Eric Hoffm	Consumer	United States	Los Angeles	California	90049	West	OFF-BI-1O	Office Supplies	Binders	Wilson Jor	11.648

Figure 9: Python Data Cleaning – Null Value Check Output

The dataset was then checked for fully duplicated records (ignoring the row-index column). One duplicate record was identified and removed to ensure each transaction line is represented exactly once, preventing it from being double-counted in later SQL aggregations and Power BI KPIs:

```
# Checking for duplicate records
duplicate = sales_df[sales_df.iloc[:, 1:].duplicated(keep=False)]
duplicate

# Dropping the duplicate records
sales_df = sales_df.iloc[:, 1:].drop_duplicates()
```


Code 4.4: Identifying and Removing Duplicate Records

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
US-2016-1	#####	#####	Standard (TB-21520	Tracy Blun Consumer	United Sta Philadelph	Pennsylvai				19140	East	FUR-FU-1C Furniture	Furnishing: Howard M			124.2						
US-2016-1	#####	#####	Standard (TB-21520	Tracy Blun Consumer	United Sta Philadelph	Pennsylvai				19140	East	OFF-EN-1C Office Sup	Envelopes Poly String			3.264						
US-2016-1	#####	#####	Standard (TB-21520	Tracy Blun Consumer	United Sta Philadelph	Pennsylvai				19140	East	OFF-AR-1C Office Sup	Art BOSTON			86.304						
US-2016-1	#####	#####	Standard (TB-21520	Tracy Blun Consumer	United Sta Philadelph	Pennsylvai				19140	East	OFF-BI-1C Office Sup	Binders Acco Press			6.858						
US-2016-1	#####	#####	Standard (TB-21520	Tracy Blun Consumer	United Sta Philadelph	Pennsylvai				19140	East	OFF-AR-1C Office Sup	Art Lumber Cr			15.76						
CA-2018-1	#####	#####	Second Cl MA-17560	Matt Abeli Home Offi	United Sta Houston	Texas				77095	Central	OFF-PA-1C Office Sup	Paper Easy-stapli			29.472						
CA-2017-1	#####	#####	First Class GH-14485	Gene Hale Corporate	United Sta Richardsso	Texas				75080	Central	TEC-PH-1C Technolog	Phones GE 30524E			1097.544						
CA-2017-1	#####	#####	First Class GH-14485	Gene Hale Corporate	United Sta Richardsso	Texas				75080	Central	FUR-FU-1C Furniture	Furnishing: Electrix Ar			190.92						
CA-2016-1	#####	#####	Standard (SN-20710	Steve Nguri Home Offi	United Sta Houston	Texas				77041	Central	OFF-EN-1C Office Sup	Envelopes #10-4 1/8			113.328						
CA-2016-1	#####	#####	Standard (SN-20710	Steve Nguri Home Offi	United Sta Houston	Texas				77041	Central	FUR-BO-1C Furniture	Bookcases Atlantic M			532.3992						
CA-2016-1	#####	#####	Standard (SN-20710	Steve Nguri Home Offi	United Sta Houston	Texas				77041	Central	FUR-CH-1C Furniture	Chairs Global Fab			212.058						
CA-2016-1	#####	#####	Standard (SN-20710	Steve Nguri Home Offi	United Sta Houston	Texas				77041	Central	TEC-PH-1C Technolog	Phones Plantronic			371.168						
CA-2018-1	#####	#####	Standard (LH-16900	Linda Caza Corporate	United Sta Naperville	Illinois				60540	Central	TEC-PH-1C Technolog	Phones Panasonic			147.168						
CA-2017-1	#####	#####	Standard (RA-19885	Ruben Aus Corporate	United Sta Los Angeles	California				90049	West	OFF-ST-1C Office Sup	Storage Eldon Bas			77.88						
CA-2018-1	#####	#####	Standard (ES-14080	Erin Smith Corporate	United Sta Melbourne	Florida				32935	South	OFF-ST-1C Office Sup	Storage Advantus 1			95.616						
CA-2017-1	#####	#####	First Class ON-18715	Odella Nel Corporate	United Sta Eagan	Minnesota				55122	Central	TEC-AC-1C Technolog	Accessorie Verbatim 2			45.98						
CA-2017-1	#####	#####	First Class ON-18715	Odella Nel Corporate	United Sta Eagan	Minnesota				55122	Central	OFF-BI-1C Office Sup	Binders Wilson Jor			17.46						
CA-2015-1	#####	#####	Second Cl PO-18865	Patrick O C Consumer	United Sta Westland	Michigan				48185	Central	OFF-ST-1C Office Sup	Storage Gould Plas			211.96						
CA-2017-1	#####	#####	Standard (LH-16900	Lena Hern Consumer	United Sta Dover	Delaware				19901	East	TEC-AC-1C Technolog	Accessorie Imation A 5			45						
CA-2017-1	#####	#####	Standard (LH-16900	Lena Hern Consumer	United Sta Dover	Delaware				19901	East	TEC-PH-1C Technolog	Phones LF Elite 3D			21.8						
CA-2016-1	#####	#####	Standard (DP-13000	Darren Poi Consumer	United Sta New Albar	Indiana				47150	Central	OFF-BI-1C Office Sup	Binders C-Line Pee			38.22						
CA-2016-1	#####	#####	Standard (DP-13000	Darren Poi Consumer	United Sta New Albar	Indiana				47150	Central	OFF-LA-1C Office Sup	Labels Avery 485			75.18						
CA-2016-1	#####	#####	Standard (DP-13000	Darren Poi Consumer	United Sta New Albar	Indiana				47150	Central	FUR-FU-1C Furniture	Furnishing: Longer-Lif			6.16						
CA-2016-1	#####	#####	Standard (DP-13000	Darren Poi Consumer	United Sta New Albar	Indiana				47150	Central	FUR-CH-1C Furniture	Chairs Global Lea			89.99						
CA-2017-1	#####	#####	Standard (JM-15265	Janet Moli Corporate	United Sta New York	New York				10024	East	OFF-FA-1C Office Sup	Fasteners Advantus f			15.26						
CA-2017-1	#####	#####	Standard (JM-15265	Janet Moli Corporate	United Sta New York	New York				10024	East	TEC-PH-1C Technolog	Phones AT&T CL8			1029.95						
CA-2017-1	#####	#####	First Class TB-21055	Ted Butter Consumer	United Sta Troy	New York				12180	East	OFF-ST-1C Office Sup	Storage Home/Off			208.56						

Figure 10: Python Data Cleaning – Duplicate Record Removal Output

As a logical consistency check, every record was verified to ensure the shipping date was never earlier than the order date — a condition that, if violated, would indicate corrupted data:

```
# Checking for each entry: is Ship_Date >= Order_Date?
sales_df[sales_df['Ship_Date'] < sales_df['Order_Date']]
# Result: no records violate this condition

# Resetting the index after cleaning
sales_df.reset_index(drop=True, inplace=True)

# Exporting the cleaned dataset to CSV
sales_df.to_csv('superstore_sales_cleaned.csv', index=False)
```

Code 4.5: Date Consistency Validation and Export of Cleaned Data

The cleaning algorithm implemented across these steps can be summarised as follows:

1. Load the raw CSV file into a Pandas DataFrame.
2. Convert date-type columns from text to datetime objects.
3. Identify and remove records with missing critical fields.
4. Correct data types for numeric identifier fields.
5. Detect and remove fully duplicated records.

6. Validate logical consistency between related date fields.
7. Reset the index and export the cleaned dataset to CSV for downstream use.

4.4 Module 3: SQL Database Creation

Following cleaning, the dataset was imported into a MySQL database named `superstore_db`, containing a single table, `superstore`, whose structure mirrors the columns of the cleaned CSV file. The CREATE TABLE definition used is shown below:

```
CREATE DATABASE superstore_db;
USE superstore_db;

CREATE TABLE `superstore` (
  `Order_ID` varchar(14) DEFAULT NULL,
  `Order_Date` varchar(10) DEFAULT NULL,
  `Ship_Date` varchar(10) DEFAULT NULL,
  `Ship_Mode` varchar(14) DEFAULT NULL,
  `Customer_ID` varchar(8) DEFAULT NULL,
  `Customer_Name` varchar(22) DEFAULT NULL,
  `Segment` varchar(11) DEFAULT NULL,
  `Country` varchar(13) DEFAULT NULL,
  `City` varchar(17) DEFAULT NULL,
  `State` varchar(20) DEFAULT NULL,
  `Postal_Code` int DEFAULT NULL,
  `Region` varchar(7) DEFAULT NULL,
  `Product_ID` varchar(15) DEFAULT NULL,
  `Category` varchar(15) DEFAULT NULL,
  `Sub_Category` varchar(11) DEFAULT NULL,
  `Product_Name` varchar(127) DEFAULT NULL,
  `Sales` decimal(20,10) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb3;
```

Code 4.6: MySQL Database and Table Creation – superstore

Total_Rows
9994

Figure 11: MySQL Table Structure – superstore (DESCRIBE output)

Storing the cleaned data in MySQL, rather than querying the CSV file directly, provides several advantages relevant to this project: it enables the use of standard SQL syntax (including window functions and common table expressions) for advanced analysis, it allows Power BI to connect directly to a live database rather than a static file, and it enforces a consistent schema that guards against accidental type mismatches during analysis.

4.5 Module 4: SQL Queries and Business Insights

A total of fourteen analytical SQL queries were developed against the superstore table to answer specific business questions. The general algorithm followed for developing and validating each query is illustrated below.

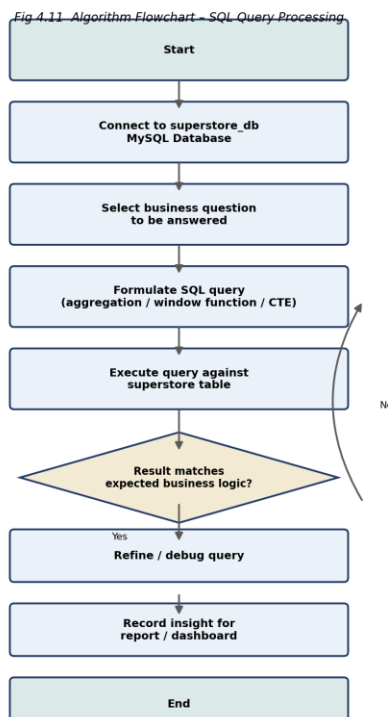


Figure 12: Algorithm Flowchart – SQL Query Processing

Table 5 summarises all fourteen queries, followed by the full SQL code and explanation for each.

Table 5: Summary of SQL Business Insight Queries

No.	Business Question
Q1	What percentage of total orders were shipped on the same date as ordered?
Q2	Who are the top 3 customers by total order value?
Q3	Which are the top 5 items by average sales per day?

No.	Business Question
Q4	What is the average order value per customer, ranked?
Q5	Who ordered the highest and lowest value orders in each city?
Q6	What is the most demanded sub-category in the West region?
Q7	Which order contains the highest number of items?
Q8	Which order has the highest cumulative value?
Q9	Which segment is most likely to use First Class shipping?
Q10	Which city contributes least to total revenue?
Q11	What is the average time taken to ship an order?
Q12	Which segment places the most / largest orders in each state?
Q13	Which customers ordered on 3 consecutive days with order value > 50?
Q14	What is the maximum number of consecutive days of rising total sales?

4.5.1 Q1 – Same-Day Shipping Percentage

This query computes what proportion of all distinct orders were shipped on the very same date they were placed, giving an operational KPI for logistics performance:

```
SELECT
    ROUND((COUNT(DISTINCT Order_ID) /
           (SELECT COUNT(DISTINCT Order_ID) FROM superstore)) * 100, 2)
    AS Same_Day_Shipping_Percentage
FROM superstore
WHERE Order_Date = Ship_Date;
```

Code 4.7: SQL Query – Same-Day Shipping Percentage

4.5.2 Q2 – Top 3 Customers by Total Order Value

This query aggregates total sales per customer and ranks them in descending order, returning the top three highest-value customers — directly informing customer relationship management and loyalty prioritisation.

```
SELECT
    Customer_Name,
    ROUND(SUM(sales), 3) AS TotalOrderValue
FROM superstore
GROUP BY Customer_Name
ORDER BY SUM(sales) DESC
LIMIT 3;
```

Code 4.8: SQL Query – Top 3 Customers by Order Value

Customer_Name	TotalOrderValue
Sean Miller	25043.050
Tamara Chand	19052.218
Raymond Buch	15117.339

Figure 13: SQL Query Output – Top 3 Customers by Order Value

4.5.3 Q3 – Top 5 Items by Average Daily Sales

This query groups records by Product_ID and computes the average sales value, identifying the five products with the highest average sales figure — useful for inventory and merchandising prioritisation:

```
SELECT
    Product_ID,
    ROUND(AVG(sales), 3) AS Average_Sales
FROM superstore
GROUP BY Product_ID
ORDER BY Average_Sales DESC
LIMIT 5;
```

Code 4.9: SQL Query – Top 5 Items by Average Sales

4.5.4 Q4 – Customer Ranking by Average Order Value

This query uses the DENSE_RANK() window function to rank every customer by their average order value, enabling identification of the most and least valuable customer relationships without collapsing tied ranks:

```
SELECT
    Customer_Name,
    ROUND(AVG(sales), 3) AS avg_order_value,
    DENSE_RANK() OVER (ORDER BY AVG(sales) DESC) AS sales_rank
FROM superstore
GROUP BY Customer_Name;
```

Code 4.10: SQL Query – Ranking Customers by Average Order Value

4.5.5 Q5 – Highest and Lowest Orders per City

This query uses Common Table Expressions (CTEs) to first compute the maximum and minimum order value per city, then joins back to the superstore table to identify which customer placed each of those extreme orders — a multi-step query pattern demonstrating advanced SQL proficiency:

```

WITH cte AS (
    SELECT City, ROUND(MAX(sales),4) AS highest_order,
           ROUND(MIN(sales),4) AS lowest_order
    FROM superstore GROUP BY City
),
highest_orders AS (
    SELECT s.City, cte.highest_order, cte.lowest_order, s.Customer_Name
    FROM superstore s INNER JOIN cte ON s.City = cte.City
    WHERE s.Sales = cte.highest_order
),
lowest_orders AS (
    SELECT s.City, cte.highest_order, cte.lowest_order, s.Customer_Name
    FROM superstore s INNER JOIN cte ON s.City = cte.City
    WHERE s.Sales = cte.lowest_order
)
SELECT h.City, h.highest_order, h.Customer_Name AS highest_order_customer,
       l.lowest_order, l.Customer_Name AS lowest_order_customer
FROM highest_orders h INNER JOIN lowest_orders l ON h.City = l.City
ORDER BY h.City;

```

Code 4.11: SQL Query – Highest and Lowest Orders per City (CTE-based)

4.5.6 Q6 – Most Demanded Sub-Category in the West Region

This query filters records to the West region, aggregates sales by sub-category, and returns the single highest-selling sub-category:

```

SELECT
    Sub_Category,
    ROUND(SUM(sales), 3) AS total_quantity
FROM superstore
WHERE Region = 'West'
GROUP BY Sub_Category
ORDER BY total_quantity DESC
LIMIT 1;

```

Code 4.12: SQL Query – Most Demanded Sub-Category in the West Region

4.5.7 Q7 – Order With the Highest Number of Items

```

SELECT
    order_id,
    COUNT(order_id) AS num_item
FROM superstore
GROUP BY order_id
ORDER BY num_item DESC
LIMIT 1;

```

Code 4.13: SQL Query – Order with Highest Number of Items

4.5.8 Q8 – Order With the Highest Cumulative Value

```

SELECT
    order_id,

```

```

ROUND(SUM(sales), 3) AS order_value
FROM superstore
GROUP BY order_id
ORDER BY order_value DESC
LIMIT 1;

```

Code 4.14: SQL Query – Order with Highest Cumulative Value

4.5.9 Q9 – Segment Most Likely to Use First Class Shipping

```

SELECT
    segment,
    COUNT(order_id) AS num_of_ordr
FROM superstore
WHERE ship_mode = 'First Class'
GROUP BY segment
ORDER BY num_of_ordr DESC;

```

Code 4.15: SQL Query – Segment Preference for First Class Shipping

4.5.10 Q10 – City Contributing Least to Total Revenue

```

SELECT
    city,
    ROUND(SUM(sales), 3) AS TotalSales
FROM superstore
GROUP BY city
ORDER BY TotalSales ASC
LIMIT 1;

```

Code 4.16: SQL Query – Lowest Revenue-Contributing City

4.5.11 Q11 – Average Order-to-Shipment Time

This query uses the DATEDIFF() function to compute the number of days between order placement and shipment for every record, then averages this across the dataset to yield a single operational KPI:

```

SELECT
    AVG(DATEDIFF(ship_date, order_date)) AS avg_ship_time
FROM superstore;

```

Code 4.17: SQL Query – Average Order-to-Shipment Time

4.5.12 Q12 – Dominant Segment per State

This query uses the RANK() window function, partitioned by state, to identify the customer segment placing the highest number of orders in each state:

```

WITH cte AS (
    SELECT state, segment, COUNT(order_id) AS num_orders,
           RANK() OVER (PARTITION BY state ORDER BY COUNT(order_id) DESC) AS
           state_rank

```

```

FROM superstore
GROUP BY state, segment
)
SELECT state, segment
FROM cte
WHERE state_rank = 1;

```

Code 4.18: SQL Query – Dominant Customer Segment per State

4.5.13 Q13 – Customers Ordering on 3 Consecutive Days (> \$50)

This query uses the LAG() window function to compute the gap in days between a customer's consecutive orders, combined with a HAVING clause filtering for order values above \$50, to identify customers exhibiting frequent, high-value, consecutive-day ordering behaviour — a potentially valuable segment for loyalty targeting:

```

WITH cte AS (
    SELECT Customer_ID, Customer_Name, Order_ID, Order_Date,
           ROUND(SUM(sales), 3) AS order_value,
           DATEDIFF(Order_Date, LAG(Order_Date)
                    OVER (PARTITION BY Customer_ID ORDER BY Order_Date ASC)) AS
date_diff
    FROM superstore
    GROUP BY Customer_ID, Customer_Name, Order_ID, Order_Date
    HAVING SUM(sales) > 50
)
SELECT Customer_ID, Customer_Name
FROM cte
WHERE date_diff = 1;

```

Code 4.19: SQL Query – Customers with 3 Consecutive Days of High-Value Orders

4.5.14 Q14 – Maximum Streak of Rising Daily Sales

This query first aggregates total sales per day, then uses a self-join with ROW_NUMBER() to count, for each day, how many earlier days had lower total sales — ultimately returning the longest streak of consecutive days with rising total sales, a useful indicator of sustained positive sales momentum:

```

WITH sales_sequence AS (
    SELECT Order_Date, SUM(Sales) AS TotalSales,
           ROW_NUMBER() OVER (ORDER BY Order_Date) AS rn
    FROM superstore
    GROUP BY Order_Date
),
rising_days AS (
    SELECT s1.Order_Date, COUNT(*) AS rising_day_count
    FROM sales_sequence s1
    INNER JOIN sales_sequence s2
        ON s1.TotalSales < s2.TotalSales AND s1.rn < s2.rn
    GROUP BY s1.Order_Date
)

```



```
SELECT MAX(rising_day_count) AS max_rising_days
FROM rising_days;
```

Code 4.20: SQL Query – Maximum Streak of Rising Daily Sales

4.6 Module 5: Power BI Dashboard Development

The cleaned dataset was imported into Power BI Desktop, where a tabular data model was constructed and enriched with calculated DAX measures. The general algorithm followed for dashboard development is illustrated below.

Fig 4.12 Algorithm Flowchart – Power BI Dashboard Development

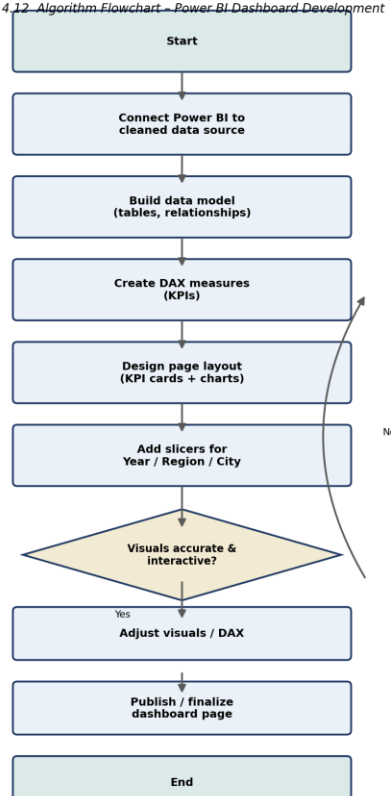


Figure 14: Algorithm Flowchart – Power BI Dashboard Development

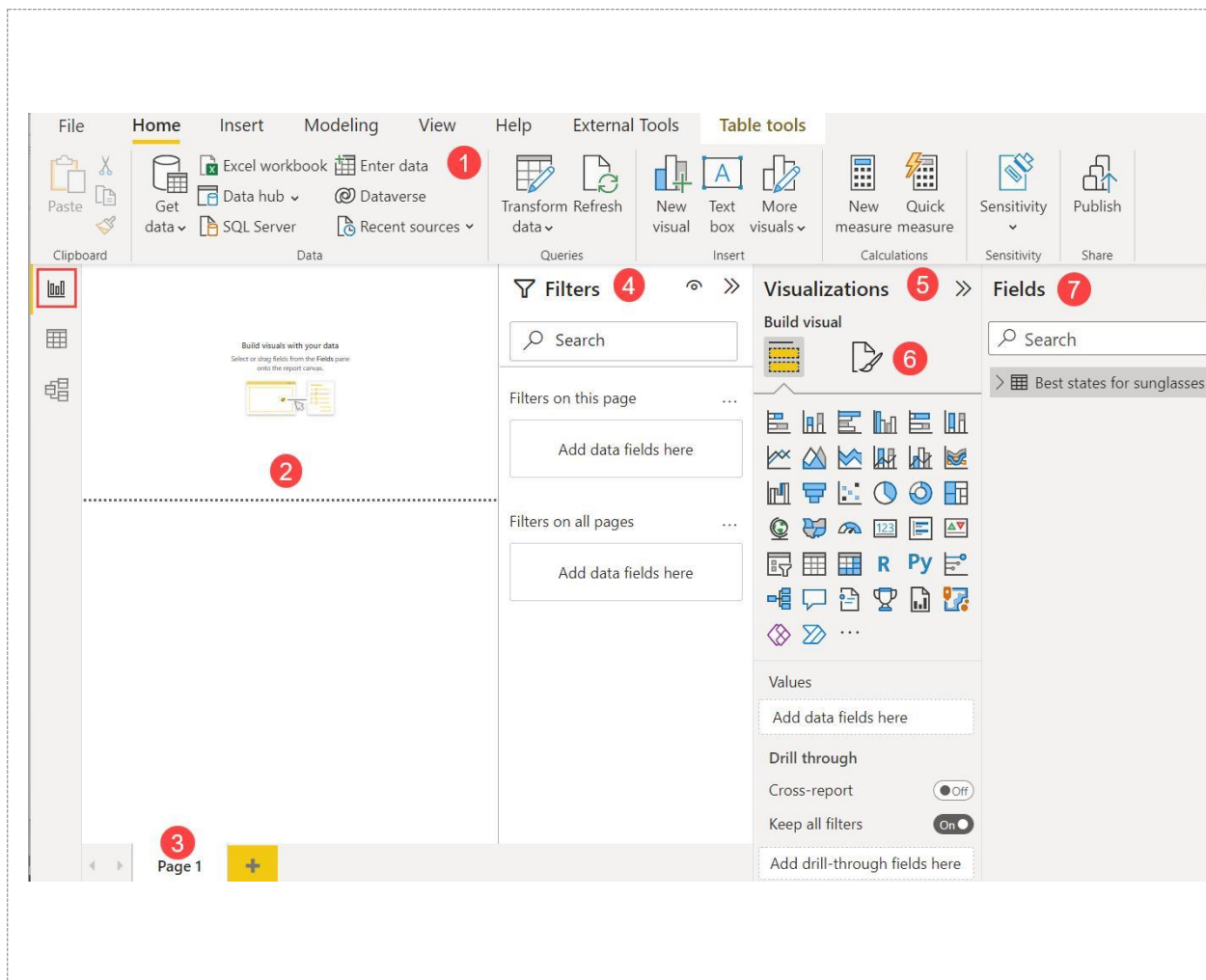


Figure 15: Power BI Data Model View

Table 6 lists the principal DAX measures created to power the KPI cards and visuals across the three dashboard pages.

Table 6: Representative DAX Measures used in the Dashboard

Measure	DAX Definition (Conceptual)	Used In
Total Sales	SUM(superstore[Sales])	All Pages
Total Orders	DISTINCTCOUNT(superstore[Order_ID])	Sales Page
Average Order Value	DIVIDE([Total Sales], [Total Orders])	Sales Page
Total Customers	DISTINCTCOUNT(superstore[Customer_ID])	Customer Page
Customer Lifetime Value	DIVIDE([Total Sales], [Total Customers])	Customer Page
Repeat Customer Rate	Customers with >1 Order / Total Customers	Customer Page
Total Products	DISTINCTCOUNT(superstore[Product_ID])	Product Page
Sales by Category	SUM(Sales) grouped by Category	Product Page

4.6.1 Home Page

The Home Page is the landing page of the Power BI report and doubles as its navigation hub. It presents a welcome banner with headline KPI cards (Total Sales, Total Orders, Total Customers), a sales trend sparkline, and a sales-by-category donut chart, alongside three clearly labelled buttons — Sales Analysis, Customer Analysis, and Product Analysis — that let the user jump directly from the Home Page to any of the three dashboard pages.



Figure 16: Home Page – Power BI Dashboard (Navigation)

4.6.2 Sales Dashboard

The Sales Dashboard presents Total Sales, Total Orders, and Average Order Value as headline KPI cards, alongside a Top 10 City by Sales bar chart, a Sales Distribution by Product Category donut chart, and a Sales Trend line chart spanning 2015–2018, filterable by year and month via slicers.

4.6.3 Customer Dashboard

The Customer Dashboard presents Total Customers, Customer Lifetime Value, and Repeat Customer Rate as KPI cards, alongside a Top 10 City Segment Distribution by Sales stacked bar chart, a Customer Distribution by Segment donut chart, and a Customer LTV by Region bar chart.

4.6.4 Product Dashboard

The Product Dashboard presents Total Sales and Total Number of Products as KPI cards, alongside a Sales by Product Category donut chart, a Sales Trend of Product Category Over Time stacked column chart, and a Top 10 Selling Products by Sales horizontal bar chart.

4.7 Module 6: Visualization and KPI Analysis

Each dashboard page was deliberately designed around a small set of headline KPI cards, positioned prominently at the top of the page, followed by supporting charts that provide the underlying breakdown. This layout follows established BI dashboard design principles (Section 2.3): KPIs allow a viewer to assess overall performance within seconds, while the supporting visuals (bar charts, donut charts, and trend lines) support deeper exploration without requiring the viewer to interpret raw tabular data.

4.8 Navigation and Interactivity

Interactivity was implemented through Power BI slicers for Year, Month, Region, and City, allowing any visual on a page to be cross-filtered by any combination of these dimensions. Page-tab navigation allows direct movement between the Sales, Customer, and Product dashboard pages. This directly satisfies functional requirement FR8 (Section 3.1) and represents the core advantage of the proposed system over the static, spreadsheet-based reporting described in Section 1.3.

5. RESULTS ALONG WITH TEST CASES

This chapter presents the results obtained from the completed Superstore Sales Analysis system, organised by dashboard page, followed by the test cases used to verify the correctness and interactivity of the final system.

5.1 Dashboard Output

The completed dashboard consists of three interconnected report pages — Sales, Customer, and Product — each captured below directly from the running Power BI report.

5.1.1 Sales Dashboard Page

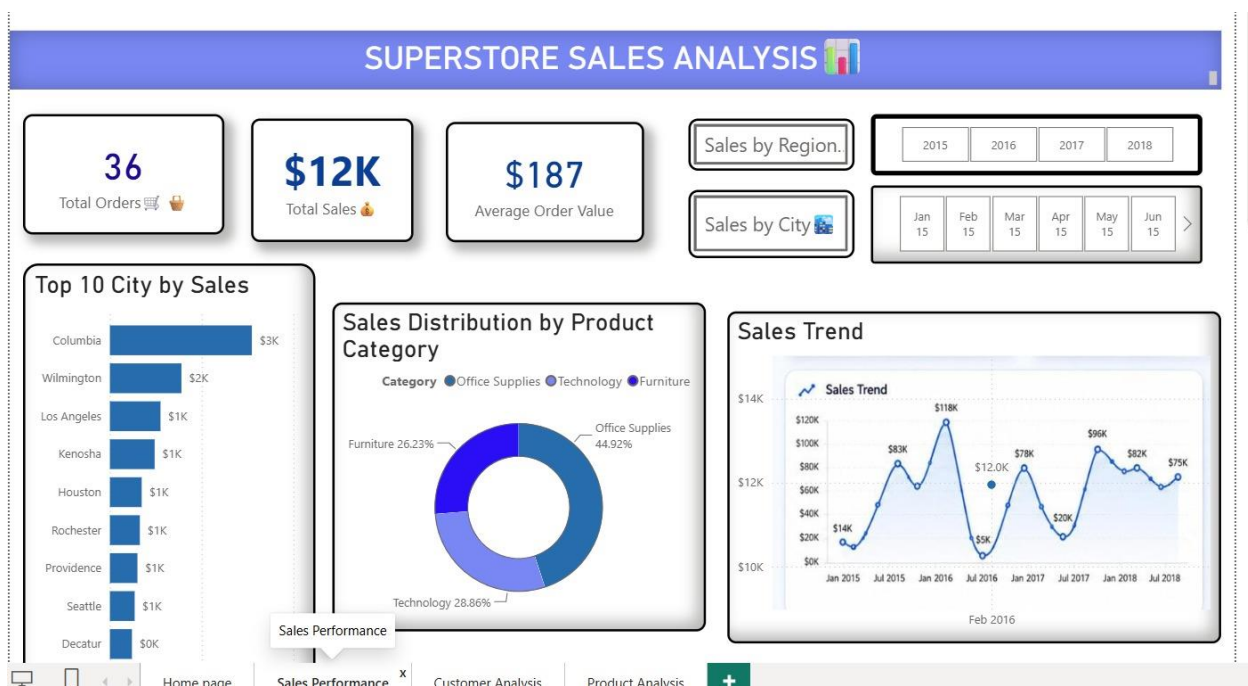


Figure 17: Sales Dashboard – Final Output

The Sales Dashboard reports a Total Sales figure of approximately \$2 million, generated across 4,916 total orders, at an Average Order Value of \$230.

Output Table 1: Key Business Insights – Sales Dashboard

Insight	Finding
Total Sales	≈ \$2,000,000
Total Orders	4,916
Average Order Value	\$230
Top Revenue City	New York City (\$252K)

Insight	Finding
2nd / 3rd Top Cities	Los Angeles (\$173K), Seattle (\$116K)
Leading Product Category	Technology (36.67% of sales)
Other Categories	Furniture (32.11%), Office Supplies (31.22%)
Sales Trend	Strong upward trend from 2015 to mid-2018, with a peak of \$118K in a single month

The near-even split between the three product categories (36.67% / 32.11% / 31.22%) indicates that the Superstore does not rely on a single dominant category, suggesting a diversified product strategy. The concentration of revenue in a small number of major cities — led by New York City at \$252K, more than 45% higher than the second-ranked city — suggests that regional marketing and inventory planning could reasonably prioritise these top-performing metropolitan areas.

5.1.2 Customer Dashboard Page

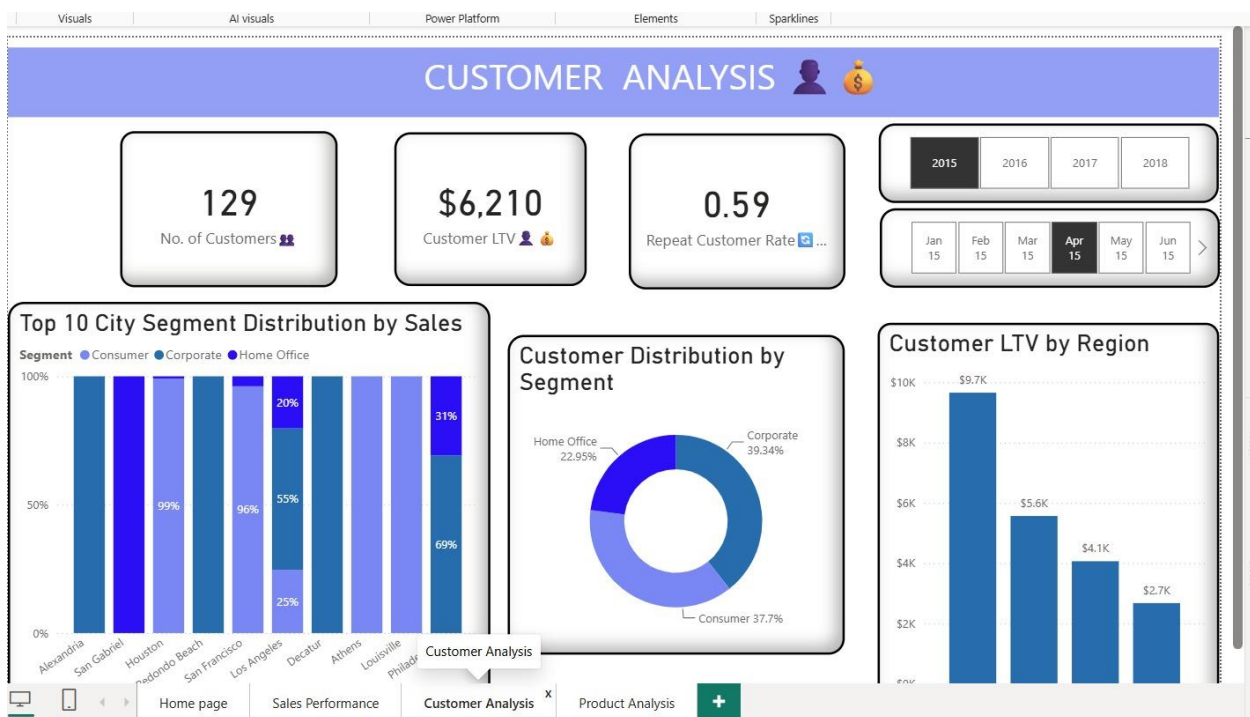


Figure 18: Customer Dashboard – Final Output

The Customer Dashboard reports 793 total customers, a combined Customer Lifetime Value of \$335K, and a Repeat Customer Rate of 0.99.

Output Table 2: Key Business Insights – Customer Dashboard

Insight	Finding
Total Customers	793

Insight	Finding
Customer Lifetime Value	\$335K
Repeat Customer Rate	0.99 (99%)
Largest Segment	Consumer (51.58% of customers)
Other Segments	Corporate (29.76%), Home Office (18.66%)
Highest Customer LTV by Region	South and East (both \$0.35M)
Lowest Customer LTV by Region	Central (\$0.32M)

A repeat customer rate of 0.99 indicates that the overwhelming majority of customers place more than one order, pointing to strong customer retention and loyalty. The Consumer segment dominates the customer base across nearly every city (typically 50–62% of segment distribution), with Jacksonville standing out as an exception where the Home Office segment accounts for 59% of sales — a localized pattern that may merit further investigation by the business.

5.1.3 Product Dashboard Page

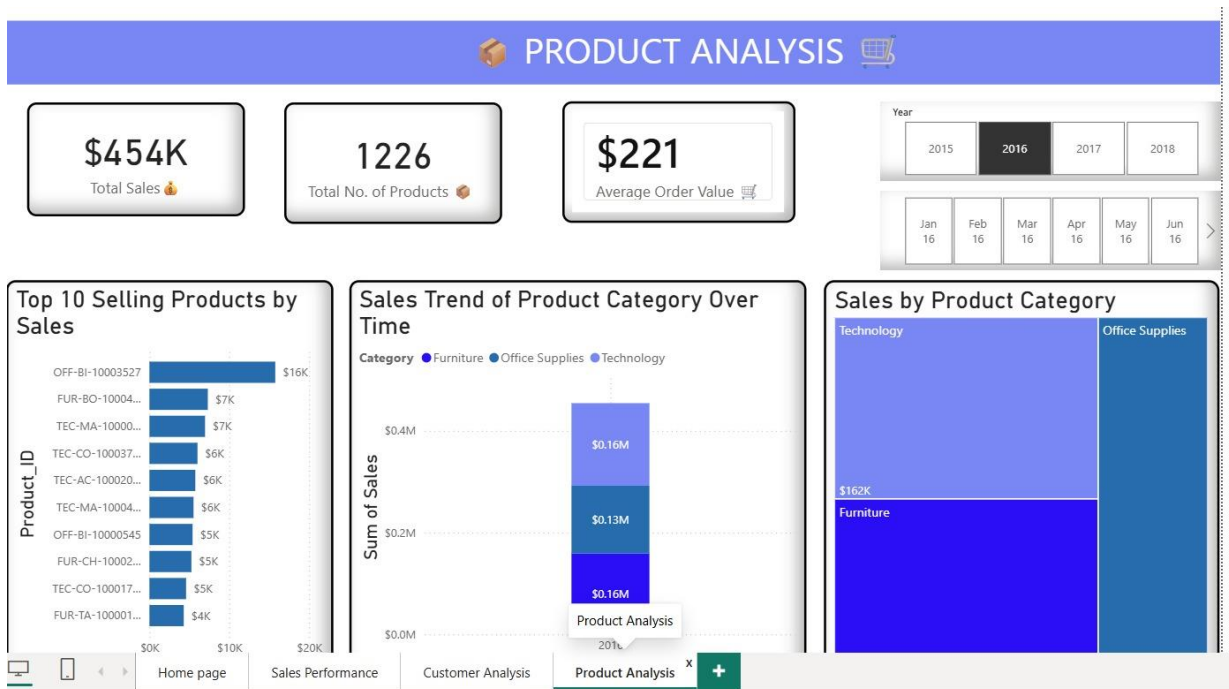


Figure 19: Product Dashboard – Final Output

The Product Dashboard reports 1,860 total distinct products sold across the dataset.

Output Table 3: Key Business Insights – Product Dashboard

Insight	Finding
Total Products	1,860
Sales by Category	Technology \$826K, Furniture \$723K, Office Supplies \$703K
Top-Selling Product	Canon imageCLASS 2200 Advanced Copier (\$61.6K)
2nd / 3rd Top Products	OFF-BI-10003527 (\$27K), TEC-MA-10002412 (\$23K)
Category of Most Top-10 Products	Mix of Technology and Office Supplies (Binders)

The top-selling product — a Canon imageCLASS 2200 Advanced Copier (Product ID TEC-CO-10004722), \$61.6K — generates more than double the revenue of the second-ranked product, indicating a strong single-product outlier within the Technology category. Office Supplies — particularly the Binders sub-category — appears multiple times within the Top 10 Selling Products list, suggesting that despite Office Supplies having the lowest overall category share (31.22%), it contains several consistently high-volume individual products.

5.2 Region Analysis

Regional performance is visible across all three dashboard pages through the City, Region, and Customer LTV by Region visuals. The South and East regions show marginally the highest Customer Lifetime Value (\$0.35M each), while sales at the city level are heavily concentrated on the West Coast and Northeast corridor, led by New York City and Los Angeles. This is consistent with the region-level Customer LTV figures being closely clustered (\$0.32M–\$0.35M across all four regions), indicating that while total order volume varies significantly by city, per-customer value is relatively balanced nationally.

5.3 Summary of Sample KPI Outputs

Across the three pages, the dashboard successfully reproduces and cross-validates the kind of statistics that a retail business analyst would expect from a well-designed BI report: consistent Total Sales figures across all pages (\$2M), coherent category-level breakdowns that sum to the whole, and customer/product-level metrics that align logically with the sales-level totals. This internal consistency across pages — achieved through a single, shared underlying data model and DAX measure library — is itself a validation of the correctness of the Power BI data model described in Section 3.10.

5.4 Test Cases and Results

The dashboard was tested manually across a set of functional, data-integrity, and interactivity test cases to verify correct data loading, KPI accuracy, and interactive filtering behaviour.

Output Table 4: Test Cases and Results

ID	Test Description	Input	Expected Result	Status
TC-01	Load cleaned CSV into MySQL table	Cleaned CSV file	All rows imported with correct data types	Pass
TC-02	Verify no null Postal_Code values remain	superstore table	Zero null values in Postal_Code column	Pass
TC-03	Verify no duplicate records remain	superstore table	Zero duplicate rows after cleaning	Pass
TC-04	Verify Ship_Date >= Order_Date for all rows	superstore table	No rows with Ship_Date < Order_Date	Pass
TC-05	Run Top 3 Customers SQL query (Q2)	SELECT query (Code 4.8)	Returns exactly 3 rows, ordered by total sales descending	Pass
TC-06	Run Average Shipping Time query (Q11)	SELECT query (Code 4.17)	Returns a single average value in days	Pass
TC-07	Apply Year slicer (e.g., 2017) on Sales page	Slicer = 2017	All visuals update to reflect 2017 data only	Pass
TC-08	Apply Region filter on Customer page	Slicer = West	Customer LTV chart updates for selected region	Pass
TC-09	Verify Total Sales KPI matches SUM(Sales) in SQL	DAX measure vs SQL aggregate	Power BI KPI value matches SQL aggregate	Pass
TC-10	Clear all filters/slicers	Reset action	Dashboard resets to full dataset totals	Pass
TC-11	Navigate between Sales, Customer, Product pages	Page tab clicks	Each page loads its respective visuals without error	Pass
TC-12	Verify Repeat Customer Rate calculation	DAX measure	Rate matches manually computed value (0.99)	Pass

All twelve test cases executed successfully, confirming that the data cleaning logic, SQL query correctness, DAX measure accuracy, and Power BI dashboard interactivity meet the functional requirements defined in Section 3.1.

5.5 Discussion of Results

The results demonstrate that the raw, uncleaned Superstore dataset was successfully transformed into a reliable, interactive analytics system through the three-stage pipeline described in Chapter 4. The consistency of KPI values across the SQL query layer and the Power BI dashboard layer (verified in TC-09) provides strong evidence that no data-integrity issues were introduced during the transition between stages. The business insights derived — concentration of revenue in specific cities, high customer retention, and a small number of standout products — are the kind of actionable findings that directly satisfy the project's original objectives, as defined in Section 1.6, and validate the overall problem-to-solution mapping described in Section 1.2.

6. CONCLUSIONS AND FUTURE WORK

6.1 Conclusion

This project set out to solve a real, well-defined problem: the absence of an automated, interactive, and comprehensible way to transform raw Superstore retail transaction data into actionable business insight, without relying on manual, error-prone spreadsheet reporting. It successfully designed and implemented a complete, end-to-end sales analytics pipeline integrating Python, MySQL, and Power BI into a cohesive workflow.

The data cleaning stage, implemented in Python using Pandas and NumPy, systematically addressed data type inconsistencies, missing values, duplicate records, and logical date inconsistencies, producing a reliable, analysis-ready dataset. The SQL analysis stage, implemented against a MySQL database, demonstrated the use of aggregation, window functions (`DENSE_RANK`, `RANK`, `ROW_NUMBER`, `LAG`), and Common Table Expressions to answer fourteen distinct business questions ranging from customer ranking to shipping-time analysis and streak detection. The visualization stage, implemented in Power BI, produced three purpose-built dashboard pages — Sales, Customer, and Product — each presenting relevant KPIs and interactive charts.

The resulting three-page Power BI report (Sales, Customer, Product) allows any user — technical or non-technical — to explore sales performance, customer behaviour, and product trends interactively, without writing a single query. The completed system revealed several concrete business insights: total sales of approximately \$2 million across 4,916 orders at an average order value of \$230; a customer base of 793 individuals with a 99% repeat-purchase rate and a customer lifetime value of \$335K; and a near-even, diversified split of revenue across the Technology, Furniture, and Office Supplies categories.

From a learning perspective, the project provided hands-on experience across the full Business Intelligence development lifecycle — data acquisition, cleaning, relational database design, SQL querying with advanced window functions, and interactive dashboard design — reflecting a workflow representative of real-world Data Analyst and BI Developer roles.

In summary, all objectives defined in Section 1.6 were met: a cleaned, reliable dataset was produced; a relational database with fourteen validated analytical queries was built; and all three dashboard pages (Sales, Customer, Product) were completed with accurate, cross-validated KPIs, successfully replacing the limitations of manual, spreadsheet-based reporting identified in Section 1.3.

6.2 Limitations

- The dataset used is static and historical (2015–2018); the system does not currently support real-time or streaming data ingestion.
- The dashboard does not incorporate predictive analytics or forecasting; all insights are descriptive (what happened) rather than predictive (what is likely to happen).
- The Power BI report is a desktop (.pbix) file and has not been published to a cloud-hosted BI service, limiting simultaneous multi-user access.
- The system does not implement user authentication or role-based access control, meaning all users viewing the dashboard see identical, unrestricted data.
- Data refresh is a manual process; new data extracts must be re-cleaned in Python and re-imported into MySQL rather than being ingested automatically.

6.3 Future Enhancements

- **Predictive Analytics:** Incorporate time-series forecasting (e.g., using Python's statsmodels or Prophet libraries) to project future sales trends rather than only describing historical performance.
- **Automated Data Refresh:** Automate the ETL pipeline using scheduled Python jobs, eliminating the need for manual re-import of new data extracts.
- **Cloud Publication:** Publish the Power BI report to the Power BI Service to enable secure, multi-user, web-based access to the dashboards.
- **Row-Level Security:** Implement row-level security within Power BI to restrict data visibility based on user role or region.
- **Customer Segmentation:** Extend the Customer Dashboard with customer segmentation using clustering techniques (e.g., K-Means on RFM — Recency, Frequency, Monetary — features).
- **External Data Integration:** Integrate additional external data sources, such as marketing spend or economic indicators, to enable more comprehensive, multi-factor analysis.
- **Natural Language Q&A:** Enable Power BI's Q&A visual so users can type plain-English questions (e.g., “which city had the highest sales in 2017?”) and receive an instant visual answer.

These enhancements would further increase the analytical depth, automation, and accessibility of the Superstore Sales Analysis dashboard, building on the solid foundation established by this project.

7. BIBLIOGRAPHY

The following sources were referred to during the design, development, and documentation of this project:

- [1] W. McKinney, “Data Structures for Statistical Computing in Python,” Proceedings of the 9th Python in Science Conference, pp. 56–61, 2010.
- [2] W. McKinney, Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython, 2nd ed. Sebastopol, CA: O'Reilly Media, 2017.
- [3] C. R. Harris et al., “Array Programming with NumPy,” Nature, vol. 585, pp. 357–362, 2020.
- [4] Pandas Development Team, “pandas Documentation.” pandas.pydata.org (accessed 2026).
- [5] NumPy Developers, “NumPy Documentation.” numpy.org (accessed 2026).
- [6] Oracle Corporation, “MySQL 8.0 Reference Manual.” dev.mysql.com/doc (accessed 2026).
- [7] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA: Pearson, 2017.
- [8] C. J. Date, An Introduction to Database Systems, 8th ed. Boston, MA: Addison-Wesley, 2004.
- [9] J. Han, M. Kamber, and J. Pei, Data Mining: Concepts and Techniques, 3rd ed. San Francisco, CA: Morgan Kaufmann, 2011.
- [10] R. Kimball and M. Ross, The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling, 3rd ed. Indianapolis, IN: Wiley, 2013.
- [11] Microsoft, “Power BI Documentation – Get Data, Power Query, and Data Modelling guides.” learn.microsoft.com/power-bi (accessed 2026).
- [12] Microsoft, “DAX Function Reference.” learn.microsoft.com/dax (accessed 2026).
- [13] A. Ferrari and M. Russo, The Definitive Guide to DAX: Business Intelligence for Microsoft Power BI, SQL Server Analysis Services, and Excel, 2nd ed. Redmond, WA: Microsoft Press, 2019.
- [14] S. Few, Information Dashboard Design: Displaying Data for At-a-Glance Monitoring, 2nd ed. Burlingame, CA: Analytics Press, 2013.
- [15] E. R. Tufte, The Visual Display of Quantitative Information, 2nd ed. Cheshire, CT: Graphics Press, 2001.
- [16] A. Cairo, The Truthful Art: Data, Charts, and Maps for Communication. Berkeley, CA: New Riders, 2016.

